

# A measured case for etching world-action-model inference into a robot's head

The ChatGPT moment for robotics requires running robot policies on the edge.

Source tags: **[M]** measured ourselves · **[S]** computed by our simulator · **[F]** measured in production at a robotics company · **[X]** published elsewhere · **[T]** a target, not yet reached.

## Executive summary

Humanoid robotics is converging on video-action diffusion transformers that predict several seconds of motion and must replan fast enough to achieve human-like reaction latency. Today's leading models are still too large to run inside a robot head, forcing teams to rely on kilowatt-class datacenter GPUs and continuous network connections. That puts latency, Wi-Fi jitter, and connectivity failures directly inside the robot's control loop

**[F]**.

Our first product targets **5× Thor's performance at the same 40 W** **[T]**. That gain would immediately support stronger robot demos, larger on-device models, lower inference cost, and more local control.

The upside grows as the model stabilizes. Rebuilding the memory system around the workload could push gains toward **~8×** **[T]**, while compute-in-memory — hardening stable weights and computation into silicon, similar to Taalas's approach for language models **[X]** — could reach **~15×** **[T]**.

## 1. The workload arrived before the chip did

State-of-the-art world-action models cannot run in real time on edge GPUs. Therefore, teams have been forced to build extraordinary infrastructure around remote inference — including low-latency video transport, datacenter accelerators, trajectory streaming, delay compensation, and local fallback control. **But remote inference can never scale beyond the demo.**

The critical system-level metric is reaction latency: the elapsed time between capturing an observation and executing the corresponding action. Human visuomotor reaction time is roughly 200 milliseconds, and delays substantially beyond this range are readily perceived as sluggishness or hesitation. For a robot, this “glass-to-action” latency includes every stage of the pipeline: sensor capture and onboard IPC or bus transfers; transmission of camera frames and proprioceptive state to the datacenter; the model’s inference loop; transmission of the generated trajectory back to the robot; and the final onboard processing required before the command reaches the control loop.

Remote inference introduces more than a fixed delay. By the time an action chunk reaches the robot, it was generated from observations that are already stale, and the amount of staleness changes from one cycle to the next. Buffering, local stabilization, and [real-time chunking](#) can smooth the resulting discontinuities , but only by trading reactivity for stability. They cannot recover changes in the world that happened after the observation was captured.

Wireless reliability creates an additional bottleneck. Wi-Fi can exhibit substantial jitter, intermittent packet loss, and abrupt degradation as the robot moves through a building. A system that performs well near an access point may fail when the robot enters a basement, passes through a doorway, or encounters network congestion. During evaluations of Sunday Robotics' autonomous ACT-2 policy in real homes, unpredictable Wi-Fi latency and network jitter reportedly emerged as major bottlenecks and failure modes, motivating a shift toward local, edge-compute control [8]. An autonomous robot cannot treat continuous datacenter connectivity as a dependable part of its control loop.

---

## 2. The bottleneck is energy efficiency, not memory bandwidth

The datacenter and the edge are solving almost opposite engineering problems. Much of modern datacenter accelerator design is organized around the **memory-bandwidth wall**. On datacenter racks, compute has grown faster than memory bandwidth, so AI workloads are typically memory-bound, which motivates optimizations around HBM, caching, batching, operator fusion, and packaging. However, on edge chips, the same workload becomes compute-bound due to power constraints. To address this bottleneck, we must focus on energy efficiency.

A DreamZero-class model makes the ratio concrete. Its 14 billion parameters across 40 layers require about  $7 \times 10^{14}$  operations per control chunk. Each step must also stream the model state that cannot fit on-die: 7.0 GB of 4-bit weights plus a 7.7 GB FP8 context window, or about 44 GB per chunk. That gives an arithmetic intensity of roughly 16,000 FLOPs per byte fetched [5].

At 40 W, Thor sustains roughly 220–300 FP4 TFLOP/s [5] against 273 GB/s of memory bandwidth [6: NVIDIA JETSON THOR SERIES MODULES DATASHEET] — a roofline ridge near 1,000 FLOPs per byte. The workload needs 16,000. Compute runs out about 15× before bandwidth does; the binding constraint is joules, not bytes.

## Why a purpose-built chip can beat a GPU at 40 W

Specialization can deliver large efficiency gains: TPUv4 achieved 1.6–3.2× higher performance per watt than A100 on measured workloads at the same process node [7: ISCA 2023]. The opportunity is reducing the overhead of programmability.

Hameed et al. found that only about 6% of a general-purpose processor’s energy reached its functional units, with much of the rest spent on clocking, caches, register files, control, and data movement [8: ISCA 2010]. Batch-one robot-policy inference is unusually easy to specialize: the graph, tensor shapes, operator order, and memory-access pattern are largely fixed, yet our workload launches roughly 1,200 GPU kernels per trajectory chunk [5]. An RPU can hard-code that schedule, stream weights predictably, keep intermediates local, and tailor the memory hierarchy to the workload.

The same overhead appears in memory traffic. WAN-class video models can become memory-bandwidth-bound even on B200 [9] because real execution moves more data than the fused minimum. An unfused attention path moves roughly 25× more data than our fused model [5], while batch-one inference must refetch weights for every request.

A high-level summary of the first-silicon design:

- **Built for batch-one continuous transformer inference.** A single always-on model gives us predictable execution, persistent state, and deterministic latency.
- **Static execution instead of GPU-style runtime machinery.** The model is compiled into a fixed hardware schedule, eliminating much of the control overhead required for arbitrary workloads.
- **A workload-specific low-precision datapath.** An output-stationary systolic array keeps partial sums local and is optimized around the numerical formats and accumulation structure these models actually use.
- **Memory movement is explicitly managed.** Compiler-managed on-chip SRAM, predictable access patterns, and reuse of weights and model state minimize expensive DRAM traffic and replace much of the cache-driven behavior of a general-purpose GPU.
- **High-traffic operations stay fused on chip.** Attention and other intermediates remain local wherever possible, avoiding unnecessary movement through external memory.
- **Core thesis:** when the model structure, precision, state reuse, and execution order are known ahead of time, the hardware should exploit that knowledge everywhere.

LEVEL

LIKELY RANGE (90%)

CEILING

WHAT UNLOCKS IT

|                        |                              |                  |                                                                                                                                               |
|------------------------|------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>First silicon</b>   | 2.7–6.7× <small>[S1]</small> | 9.1×             | Static schedule; FP4 systolic linear path; dedicated FP8 attention; guidance weight sharing; fused attention; weight-streaming memory system. |
| <b>Mature compiler</b> | 3.9–9.7× <small>[S1]</small> | 15.7×; 22× gated | Same silicon; better utilization, fusion, DMA overlap, scheduling, and clock gating.                                                          |
| <b>Model-resident</b>  | 6.0–9.7× <small>[T1]</small> | ≤ 20–44×         | Model resident in 3D-DRAM stacked onto the compute die                                                                                        |
| <b>Codesign</b>        | 7–19× <small>[T1]</small>    | ~35–47×          | Harden stable weights and operator chains while preserving programmability where the model still changes.                                     |

## First-silicon specs [T1]

- 1.9 M FP4 MACs at ~1.05 GHz (4 PF dense) in 128×128 weight-streaming systolic tiles; the model widths divide the array exactly (5120 = 8 × 640, 13,824 = 8 × 1728).
- FP8 multiply into FP32 accumulators in dedicated SRAM beneath the array — TPUv1’s narrow-multiply, wide-accumulate pattern [X1].
- 256-bit LPDDR5X interface at 307 GB/s, at least 16 GB; the prefetcher is a counter, because the chip walks the same ~44 GB address sequence every chunk.

- ~90 MB SRAM for stream buffers and the activation spine — no resident weights.
- KV ring in DRAM, advanced by pointer at chunk end; both guidance branches share each fetched weight and context block.
- Per-bank DRAM refresh in known idle windows — left unmanaged, hot-head refresh costs 14–28% of effective bandwidth and adds tail latency [5].
- No operating system, zero instruction fetch, no kernels. A GPU launches ~1,200 kernels per chunk; the RPU executes one fixed hardware schedule. TPUv1 showed what this predictability is worth under a latency bound: 80% of peak at 7 ms p99 versus 37% for a same-generation K80 [X: ISCA 2017].
- Schedule ROM with one-, two-, and three-step modes, generated offline by the calibrated simulator with a worst-case deadline certificate; weights, scales, token counts, and schedules load from an image — tile geometry and number formats stay fixed in the mask.
- ~2% of the datapath programmable for flow-ODE or CEM-style planning; a JEPA-family workload costs 1.9% of DreamZero-path performance per watt, and both stream ~7 GB, so one memory tier serves both [5].
- No further conventional-architecture multiplier. Grids of small cores, reconfigurable fabrics, and exposed datapaths preserve more flexibility but pay for it in measured efficiency [5].

## Model-resident specs [1]

- Memory stacked directly on compute cuts each weight read from  $\sim 40$  to  $\sim 8$  pJ/byte and moves the roofline that otherwise caps compute gains near  $2\times$ .
- Bound rises to  $\sim 20\text{--}31\times$ , or  $29\text{--}44\times$  with gated circuit techniques.
- Three questions gate the bet: does the stack survive a fanless head, can we buy the capacity, and do hybrid bonds tolerate vibration?
- Deepens our commitment to transformers — so it stays out of first silicon.

## Codesign specs [17]

- The ceiling is arithmetic, not a design claim: put every joule of the 40 W into FP4 multiply-accumulate at its floor ( $\sim 0.0038$  pJ/FLOP) against Thor's achieved  $0.13\text{--}0.18$ , and you get  $35\text{--}47\times$  — the most any FP4 chip can win at equal power.
- Past that, gains must come from the model, not the silicon.

## The memory-bandwidth margin

Memory bandwidth is the next bottleneck if we succeed at making compute more efficient. The current design sits only about  $1.5\times$  on the compute side of the roofline crossover (vs  $4\times$  against Thor's roofline), and hot-token refresh eats into that margin. Because we have not yet calibrated the simulator against a memory-bound workload, a moderate error in traffic accounting could move the chip across that boundary [15].

Fortunately, bandwidth is a conventional problem to solve. Lossless weight-stream compression reduces traffic to  $62\text{--}83\%$ , while widening LPDDR5X

to 512 bits provides roughly **614 GB/s**; Apple already ships 546 GB/s on-package [13]. Even a conservative FP8 case streaming ~15.5 GB per step remains below modeled compute time [15]. We can also trade MAC area for more memory channels or use short-reach on-package memory to reduce cost per bit [15]. And the model-resident design in the technical roadmap is the long term structural fix: stacking the model in 3D-DRAM on the compute die multiplies effective bandwidth while cutting energy per byte.

---

#### 4. Addressing Jensen's Law

Jensen's Law asserts that the incumbent's hardware improves by roughly **2× a year** (which is way too generous as an expectation, but valid as an upper bound). To address it, we first have to ask where those gains come from. Most of them are advances that benefit everyone: better silicon, denser SRAM, faster DRAM, improved packaging, and lower-voltage transistors. We ride the same wave as the incumbents.

Consider the TPU. Since 2013, GPUs have improved by roughly **100×**, but the TPU never needed to target 100× up front. It only had to be consistently a few times better at every stage — a multiplicative architectural factor on top of the same technology curve — which it achieves purely through the topology of the chip.

In our view the right framing is two questions: given access to the same physical primitives, what is our chip's performance multiplier? And what multiplier makes people willing to switch? The TPU shows it can be as low as **3×**; Groq's LPU and Etched suggest it may be closer to **10×**.

Our goal is the same: to stay consistently some multiple better than the incumbent at every stage, riding the same wave of general improvement. We target a modest  $5\times$  for our first product [ET1]; the long-term vision is to co-design the RPU once model architectures converge, reaching  $>15\times$  in energy efficiency [ET2].

We believe these multipliers can emerge purely from removing the generality tax of GPUs, which spend **less than 10%** of their energy on arithmetic [EX: ISCA 2010] and more energy than necessary on each operation. Energy efficiency is the product of these two factors, and it is the right metric because edge inference is bottlenecked by compute, not memory bandwidth — and diffusion transformers are more compute-dense than autoregressive LLMs.

---

## 5. Win the transition to edge inference

Full edge inference will require model efficiency and hardware efficiency to improve together. As robot policies become smaller through distillation, quantization, token reduction, caching, and better architectures, our current RPU design will still remain several steps ahead of general-purpose platforms at every stage of that transition. That near-term lead wins deployments and gives us the workload knowledge, software stack, and customer relationships needed for the larger opportunity. When the model architecture finally stabilizes, we will be positioned to co-design the model and chip as a single system — creating an accelerator that NVIDIA's general-purpose architecture cannot match.

This has a precedent. A few years ago, datacenter LLM architecture largely converged, and a market opportunity opened up for Etched's transformer

ASIC and Taalas’s approach of hardening a model directly into the fabric . Robotics is on the same trajectory, but a few years behind. The same market opportunity will open at the edge, where none of the datacenter incumbents are looking.

---

## References

1. NVIDIA, “Jetson Thor Series Modules Datasheet” (T5000: dense FP4 1,035 TFLOP/s at MAXN; 273 GB/s LPDDR5X).
2. DreamZero authors, “World Action Models are Zero-shot Policies,” arXiv:2602.15922, 2026.
3. Q. Shen et al., “World Action Models: A Survey,” arXiv:2606.20781, 2026.
4. N. Jouppi et al., “In-Datacenter Performance Analysis of a Tensor Processing Unit,” ISCA 2017.
5. N. Jouppi et al., “TPU v4: An Optically Reconfigurable Supercomputer,” ISCA 2023.
6. R. Hameed et al., “Understanding Sources of Inefficiency in General-Purpose Chips,” ISCA 2010.
7. FLASH-D, arXiv:2505.14201; ExpMul, arXiv:2505.14314.
8. WorldCache (content-aware caching for video world models), arXiv:2603.22286; Chorus (inter-request caching reuse), arXiv:2604.04451; Sliding Tile Attention, arXiv:2502.04507.
9. K. Black, M. Y. Galliker, and S. Levine (Physical Intelligence), “Real-Time Execution of Action Chunking Flow Policies,” arXiv:2506.07339, 2025.
10. Efficient-WAM (1 B world-action model), arXiv:2606.10040; LaWAM (latent world action models), arXiv:2606.15768; NanoVLA, arXiv:2510.25122.

